Lecture 9

17 October 2026 22:34

📌 Andrej Karpathy's Notes on Attention

Note 1: Attention is a communication mechanism

Think of attention as a **directed graph**:

- Every token is a **node** in the graph.
- Every edge has a weight ([Equation] " determining how much information flows along it.)"
- Tokens communicate by sending their Values (V) along these edges to update each other.

Note 2: Attention has no notion of space/order by itself

Attention operates over a **set of vectors**, not a sequence:

- If you shuffle the order of the words in the sentence, the attention mechanism calculates the **exact same affinities** between pairs of words.
- It does **not** know who is first, second, or fifth.
- That is why **Positional Embeddings (pos_emb) are strictly necessary**—they inject spatial information into x before attention runs.

Note 3: Elements across the batch dimension never talk

Every batch element is an isolated universe:

- Tokens in sequence 1 never communicate with tokens in sequence 2.
- Batch elements are processed in pure parallel: shape [Equation] preserves independence across B.

Note 4: "Autoregressive / Causal" vs. "Bidirectional" Attention

- **Decoder blocks (our language model):** Tokens are only allowed to talk to **past** tokens because we are predicting the future. We enforce this with the triangular mask (tril with -inf). This is called **causal** or **masked self-attention**.
- **Encoder blocks (e.g., BERT, sentiment classification):** Tokens *are* allowed to look into both the past and future (e.g., to understand the sentiment of an entire sentence). To make an encoder, you simply **delete the tril mask**—all tokens talk to all tokens!

Note 5: "Scaled" Dot-Product Attention (Why divide by [Equation] ?)

Why do we do * (head_size ** -0.5)?

- If Q and K have unit variance ([Equation]), their dot product [Equation] will have variance equal to [Equation] "(e.g., 16 or 64). "
- With large variance, some dot-product numbers become very large positive or negative (e.g., [Equation] or [Equation]).
- When you pass large numbers into softmax, it **saturates**: it turns into an extreme one-hot distribution (e.g., [Equation]).
- When softmax saturates, **gradients vanish** to near zero, and learning stalls.
- Dividing by [Equation] scales variance back down to 1.0, keeping softmax diffuse and gradients flowing smoothly!

Note 6: Self-Attention vs. Cross-Attention (Deep Dive)

This is one of the most important conceptual differences in Transformers:

Feature	Self-Attention (What we are building)	Cross-Attention
Where do [Equation] come from?	All three come from the **exact same source** x	Q comes from sequence x, but K and V come from an **external source** y
Typical Use Case	GPT, text generation, language modeling	Translation (e.g., French → English), Whisper (Audio → Text), Image Captioning
Who is talking to whom?	Tokens within the sequence talk to *themselves*	Tokens in one sequence ask questions to *another sequence*

1. Self-Attention in Code & Concept

In **Self-Attention**, the input sequence is reading and refining itself:

```
python
# Everything is derived from the SAME input x
q = self.query(x)  # from x
k = self.key(x)    # from x
v = self.value(x)  # from x
wei = q @ k.T
out = wei @ v
```

- **Example:** In *"The animal didn't cross the street because it was too tired"*, the token "it" looks at other tokens in the **same sentence** to figure out that "it" refers to "animal".

2. Cross-Attention in Code & Concept

In **Cross-Attention**, you have two separate sequences (e.g., English decoder and French encoder):

Encoder (French text) ————> creates Keys (K) & Values (V)

|
▼
Decoder (English output) ———> creates Queries (Q)

```
def forward(self, x, context):
  # x is the decoder's current English generation
  # context is the encoder's French sentence

  q = self.query(x)       # English tokens ask: "What does the French sentence say?"
  k = self.key(context)    # French tokens advertise their meaning
  v = self.value(context)  # French tokens provide the actual information

  wei = q @ k.T
  out = wei @ v
```

Example: The English decoder is about to predict the next word in English, so it queries the French encoder's keys and values to pull in the appropriate translated meaning.

Takeaway for Version 4:

Since we are building a standalone **GPT-style Decoder Language Model**, we are using **Causal Self-Attention**:

- **"Self"**: [Equation] all come from [Equation] .
- **"Causal"**: Future tokens are masked with tril.

If there is confusion between using C or the head size -----> refer this :

Question ask to antigravity:
in the video sir has used C, and in our code we have used the head size, and in my code i have written C in the comment with mentioning 16 as its size, we know that the outout is (4, 8, 32) which we have converted into 16 and 16

1. C vs head_size

In the video, Karpathy writes:

```
python
B, T, C = 4, 8, 32   # Batch, Time, Channels
head_size = 16
key   = nn.Linear(C, head_size, bias=False)  # 32 → 16
query = nn.Linear(C, head_size, bias=False)  # 32 → 16
value = nn.Linear(C, head_size, bias=False)  # 32 → 16
```

Here:

- **C (Channels) = n_embd = 32**: This is the **original embedding size** of the token.
- **head_size = 16**: This is the **smaller subspace size** that each head projects into.

So when x has shape (B, T, C) = (4, 8, 32):

- q = query(x) turns shape (4, 8, 32) [Equation] **(4, 8, 16)**
- k = key(x) turns shape (4, 8, 32) [Equation] **(4, 8, 16)**
- v = value(x) turns shape (4, 8, 32) [Equation] **(4, 8, 16)**

2. Why you mentioned "16 and 16"

Your intuition about **"converting into 16 and 16"** is spot on in two ways:

1. **Inside the single head right now:** Both Query and Key are projected down to **16**: [Equation] The two 16 dimensions contract in the dot product!
2. **In the next step (Multi-Head Attention):** We will make **two separate heads**:
   - Head 1 gives an output of shape (4, 8, 16)
   - Head 2 gives an output of shape (4, 8, 16)
   - We concatenate them side-by-side: [Equation] And we get our full **C = 32** back!

Summary Table

Term in Video	Value	What it is
C / n_embd	32	The input channel/embedding dimension
head_size	16	The internal projection dimension for each head
q, k, v	(B, T, 16)	Vectors living in the 16-dim attention subspace
2 heads concatenated	16 + 16 = 32	Glued back to restore the full C = 32

So having C = 32 as input and projecting to 16 for Q, K, and V is exactly right!

Right now in the video, Karpathy is in the **toy scratchpad / playground cell**:

- He created a dummy random tensor: x = torch.randn(4, 8, 32).
- There is **no lm_head** and **no training loop** in this cell yet.
- The only goal in this step is to inspect the standalone math of: [Equation]

Where lm_head comes back:

Once he finishes explaining the standalone mechanics and notes in that cell:

1. He takes this Head class.
2. He pastes it into the main model (BigramLanguageModel).
3. Inside forward(), the data pipeline becomes:

[Equation]

usee of the register line :

The line:
python
self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))
tells PyTorch:

> *"Hey, keep this tensor inside the model, but **do NOT calculate gradients for it** and **do NOT let the optimizer update it**."*

Why not just do self.tril = torch.tril(...)?

If you just write self.tril = torch.tril(...), your code will break the moment you move to GPU! Here are the **2 main reasons** we use register_buffer:

reasons we use register_buffer:

1. Automatic GPU/CPU movement (.to(device)) 🚀

- If you do model.to('cuda'), PyTorch automatically moves all parameters **and all registered buffers** to your GPU.
- If you used a plain variable (self.tril = ...), it would **stay stuck on the CPU**.
- Then in forward(), when you try to mask wei (which is on GPU) with self.tril (which is on CPU), PyTorch will crash with:
  RuntimeError: Expected all tensors to be on the same device, but found cpu and cuda:0!

2. Saved with the model (state_dict) 💾

- When you save your model using torch.save(model.state_dict(), 'model.pt'), registered buffers are **saved along with the model weights**.
- When someone loads the model later, tril is automatically restored with it.

Summary: The 3 Types of Tensors in PyTorch

Method	Trained by Optimizer?	Moves to GPU with model.to(device)?	Saved in state_dict?
nn.Parameter (weights, biases)	**Yes**	**Yes**	**Yes**
register_buffer (tril)	❌ **No**	**Yes**	**Yes**
Plain Python attribute (self.x)	❌ **No**	❌ **No**	❌ **No**

Because tril is a **constant mask of 1s and 0s** (not something we want to learn), register_buffer is the exact, standard PyTorch way to store it!

Now, the whole attention head is created which is this :

```
class Head(nn.Module):
      # One single head  of self-attention

   def __init__(self, head_size):
      super().__init__()

      # Remember n_embd = C (channels) = 32, which we get after tok_emb-->+pos_emb = give (4,8,32)-->lm_head gives logits
of shape(4, 8, 65)
      self.key = nn.Linear(n_embd, head_size, bias=False)   # (4, 8, 32)  change made  32 --> 16
      self.query = nn.Linear(n_embd, head_size, bias=False)  # (4, 8, 32)  change made  32 --> 16
      self.value = nn.Linear(n_embd, head_size, bias=False)  # (4, 8, 32)  change made  32 --> 16
      self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))

   def forward(self, x):
      B, T, C = x.shape

      k = self.key(x)   # (B,T,C=16)   # the C size is changed from 32 to 16
      q = self.query(x)  # (B,T,C=16)
      # compute attention scores ("affinities")
      wei = q @ k.transpose(-2, -1) * k.shape[-1]**-0.5  # (B, T, T)
      wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))   # (B, T, T)
      wei = F.softmax(wei, dim=-1)   # (B, T, T)
      # perform the weighted aggregation of the values
      v = self.value(x)  # (B, T, C=16)
      out = wei @ v       # (B, T, T) @ (B, T, C) --> (B, T, C=16)
      .....

      return out
```

2. Changes in the BigramLanguageModel :

1. Apply the head module
self.sa_head = Head(n_embd) --> as we have to pass the head_size in the self-attention head (take a look) --> we pass n_embd

2. In forward loop :
x = tok_emb + pos_emb
x = self.sa_head(x)

3. In generated loop :
Idx_cond = idx[:, -block_size:]
The only reason why we take this is cause, in the iteration loop upto 500 (we have set max_generate_tokens = 500) so, when loop continues and generate new token let's say 25, so that new total 25 tokens needed to have a new position naming, we we have given only the raneg upto 7 indeces i.e. 0, 1, 2, …. Upto 7, so instead of updating the index, we will let the new token be generated and taken only the latest last 8 token as it is going to add (concatinated in the idx line i.e. this idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)

So, taking the latest 8 new tokens is good rather then moving those new token in some other module

and every time taking from those

Antigravity answer :

The line:

python
idx_cond = idx[:, -block_size:]
is **cropping the context window** so it never exceeds block_size (e.g. 8).

Why is this needed? (What happens without it?)

During generation, we keep appending new tokens in a loop:

python
idx = torch.cat((idx, idx_next), dim=1)

- Loop 1: sequence length [Equation]
- Loop 8: sequence length [Equation]
- Loop 9: sequence length [Equation] ... and it keeps growing up to 500 tokens!

If you pass T = 9 to the model when block_size = 8, **PyTorch will violently crash** for two reasons:

1. The Position Embedding Table runs out of indices
Remember how we defined position embeddings:

python
self.position_embedding_table = nn.Embedding(block_size, n_embd) # size = 8
This table only has **8 slots** (index 0 through 7).
In forward():

python
pos_emb = self.position_embedding_table(torch.arange(T, device=device))

IMP

- If [Equation] : indices are [0, 1, 2, 3, 4, 5, 6, 7] [Equation] Works!
- If [Equation] : indices are [0, 1, 2, 3, 4, 5, 6, 7, 8] [Equation] ❌ **CRASH!**
  IndexError: index 8 is out of bounds for dimension 0 with size 8

2. The tril ~~mask is only (block_size, block~~_size)

python
self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))
If block_size = 8, tril is an [Equation] matrix. It physically cannot mask a [Equation] attention matrix!

What idx[:, -block_size:] does

In Python, negative indexing slices from the end:

If idx has 5 tokens: idx[:, -8:] --> takes all 5 tokens (no problem)
If idx has 20 tokens: idx[:, -8:] --> takes ONLY the last 8 tokens

It creates a **sliding window**:
[token_1, token_2, token_3, ... token_48, token_49, token_50]
└──────────┬──────────┘
Takes last 8 tokens
(idx_cond)
The model only ever looks at the **most recent 8 tokens** of context to predict token 51, ensuring [Equation] is never greater than block_size!

V IMP : summary

If you want to keep a polished, crisp version in your notes:

Why idx_cond = idx[:, -block_size:]?
As generation loops up to 500 tokens, the sequence length [Equation] keeps growing (e.g., [Equation]). However, our positional embedding table only knows indices [Equation] to [Equation] (block_size = 8).
Instead of breaking or changing the table, we keep the entire generated text in idx, but feed only the **latest 8 tokens** (idx_cond) into the model to predict the next character. It acts as a sliding context window.